

Análise Comparativa de Arquiteturas para Classificação de Textos

Carlos da Mota Bergueira pg11605

Diego Jefferson Mendes Silva pg59999

Filipa Araújo Pereira pg42201

Rui Manuel Martins Marques Rodrigues pg7942

Universidade do Minho, Braga, Portugal

Abstract. Este trabalho aborda o problema da classificação da proveniência textual num cenário multiclasse, distinguindo entre textos escritos por humanos e textos gerados automaticamente, identificando, neste último caso, a família do modelo gerador (Anthropic, Google, Meta ou OpenAI). Para esse fim, foi definido um pipeline supervisionado, que inclui a recolha e integração de múltiplos *datasets* públicos, o balanceamento de classes e a divisão estratificada entre treino e avaliação. Na fase de pré-processamento, foram exploradas representações vetoriais baseadas em TF-IDF - *Term Frequency-Inverse Document Frequency* (a nível de palavra e de carácter), complementadas com características estilométricas e estatísticas. Do ponto de vista metodológico, foram comparadas quatro famílias de modelos implementadas no projeto: uma rede neuronal construída de raiz em NumPy, uma arquitetura densa em PyTorch, um Transformer encoder treinado para a tarefa e um modelo pré-treinado RoBERTa ajustado por *fine-tuning*. A comparação foi realizada com base em métricas de classificação multiclasse, com ênfase na *accuracy* e no *F1-macro*, analisando também a estabilidade por classe. Como principal contribuição, este estudo apresenta uma análise comparativa entre arquiteturas com diferentes níveis de complexidade, discutindo o compromisso entre custo computacional, capacidade representacional e robustez na classificação da proveniência textual.

Keywords: Aprendizagem Profunda, Classificação de Texto, Redes Neurais, Processamento de Linguagem Natural, Aprendizagem Automática, Inteligência Artificial

Código e reprodutibilidade. O código-fonte, instruções de execução e artefatos experimentais estão disponíveis em: <https://github.com/MIA-CDFR/AP>.

1 Introdução

A classificação de textos é uma tarefa fundamental em *Natural Language Processing* (NLP), com aplicações que vão desde a filtragem de *spam* até à análise de sentimentos e detecção de autoria. Com o aparecimento de modelos de linguagem cada vez mais sofisticados, a distinção entre textos gerados por humanos e por máquinas tornou-se um desafio crescente.

Este trabalho tem como objetivo desenvolver e comparar modelos de aprendizagem supervisionada para classificar textos segundo a sua origem, distinguindo produção humana de produção automática. Para textos de origem automática, o trabalho visa identificar a origem tecnológica do modelo gerador, considerando especificamente os ecossistemas da Anthropic, Google, Meta e OpenAI.

Para avaliar diferentes estratégias de representação textual e arquiteturas de classificação, foram desenvolvidos quatro modelos distintos: dois modelos *Deep Neural Network* (DNN) (um com implementação NumPy [1] pura e outro com PyTorch [2]), e dois modelos baseados em Transformer (um *encoder* genérico treinado de raiz e outro pré-treinado baseado em *fine-tuning* de RoBERTa). Esta abordagem permite análise comparativa entre arquiteturas vetoriais e modelos contextuais modernos de NLP.

DNN com NumPy. Implementação de uma rede neuronal profunda multiclasse com múltiplas camadas ocultas e funções de ativação não lineares, desenvolvida de raiz utilizando apenas NumPy. O modelo inclui mecanismos de controlo de *overfitting* (*dropout*, *early stopping*) e técnicas de otimização por *Stochastic Gradient Descent* (SGD).

DNN com PyTorch. Implementação de um classificador denso hierárquico, integrando uma *secondary head* para distinguir textos humanos de textos gerados por IA e um módulo de classificação de famílias para identificar a origem do modelo nas amostras classificadas como artificiais. A formulação privilegia a flexibilidade de treino e o uso eficiente da GPU (*Graphics Processing Unit*).

Transformer Genérico. Implementação de um modelo *Transformer encoder* para classificação de texto, com *embeddings* treináveis, *Multi-head attention* e camadas *feedforward*, treinado de raiz para a tarefa de classificação.

RoBERTa Fine-tuned. Utilização do modelo pré-treinado RoBERTa-base [3] para classificação, realizando *fine-tuning* específico para a tarefa de classificação de origem de textos, aproveitando conhecimento de linguagem pré-existente.

2 Seleção de *Datasets* e Pré-processamento

2.1 Fontes de Dados

Para a construção dos conjuntos de treino e teste, foram selecionados *datasets* públicos do *Hugging Face* que cobrem diferentes origens textuais: produção humana e produção automática por diferentes famílias de modelos de linguagem. A seleção teve como objetivo garantir representatividade por classe e diversidade de estilos textuais.

Os dados utilizados derivam das seguintes fontes:

MLNTeam-Unical/OpenTuringBench [4]. Fornece textos gerados pelos modelos da família Meta e Google, com 37.283 amostras por classe, totalizando 74.566 amostras. O *dataset* completo contém 5 classes adicionais (Intel, Microsoft, Mistral AI, Qwen, Upstage) com 186.415 amostras, que não foram utilizadas.

Dahoas/instruct-synthetic-prompt-responses [5]. Fornece textos gerados por modelos da família OpenAI, com 33.203 amostras.

artem9k/ai-text-detection-pile [6]. Fornece textos de origem humana, com 1.028.146 amostras.

Anthropic/persuasion [7]. Fornece textos gerados por modelos da família Anthropic, com 3.360 amostras.

Dados adicionais foram fornecidos durante o desenvolvimento experimental:

Dataset de Exemplo. Disponibilizado pelo professor na fase inicial, contendo 125 amostras com distribuição: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52). Utilizado na 1ª submissão como parte do treino.

Dataset Revelado 1ª Submissão. *Dataset* com *labels* revelados após a 1ª submissão, contendo 100 amostras com distribuição: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14). Integrado ao treino na 2ª submissão.

Dataset Revelado 2ª Submissão. *Dataset* com *labels* revelados após a 2ª submissão, contendo 100 amostras com distribuição: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16). Integrado ao treino na 2ª submissão.

Face ao acentuado desequilíbrio entre classes no conjunto de dados bruto - com 1.028.146 amostras da classe Human contra 3.360 da classe Anthropic - foi adotada uma estratégia de balanceamento no conjunto de treino de cada submissão, visando mitigar o enviesamento a favor da classe majoritária. Este procedimento combinou o *undersampling* da classe Human com o *oversampling* das classes minoritárias. Adicionalmente, o número de amostras por classe foi fixado em 5.000, resultando num conjunto balanceado com 25.000 instâncias no total por versão de treino, conforme ilustrado na Fig. 1.

Para a avaliação dos modelos, utilizou-se uma estratégia de divisão 80/20, reservando 80% dos dados para treino e 20% para teste. Esta divisão foi realizada de forma estratificada, garantindo que a proporção de amostras por classe se mantivesse uniforme em ambos os conjuntos.

2.2 Dataset de Validação

O *dataset* de validação variou conforme as diferentes submissões:

1ª Submissão. 125 amostras do *dataset* de exemplos fornecido pelo professor, distribuídas da seguinte forma: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52).

2ª Submissão. 100 amostras do *dataset* decorrentes da 1ª submissão, distribuídas da seguinte forma: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14).

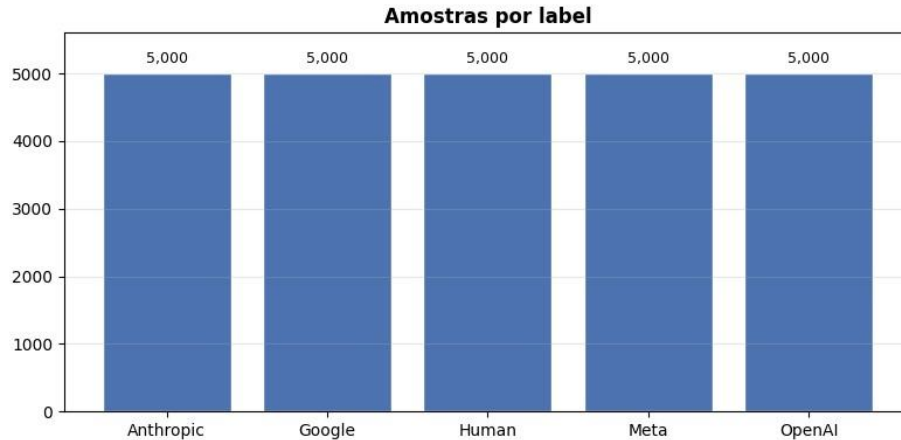


Fig.1. Distribuição das classes no conjunto de treino e teste

3ª Submissão. 100 amostras do *dataset* decorrentes da 2ª submissão, distribuídas da seguinte forma: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16).

2.3 Pré-processamento

O pré-processamento de texto foi adaptado conforme o tipo de modelo utilizado, considerando as diferentes estratégias de representação textual e os requisitos computacionais de cada arquitetura.

DNN com NumPy. O modelo DNN implementado com NumPy utiliza uma abordagem baseada em representações vetoriais híbridas, combinando TF-IDF de palavras e caracteres. O código também suporta *features* manuais estilométricas, embora essa opção não esteja ativada na configuração principal reportada.

O pré-processamento inclui:

- **Representação TF-IDF.** O texto é convertido em vetores TF-IDF de dois níveis de análise. A análise ao nível de palavras utiliza n-gramas unigrama e bigrama, enquanto a análise ao nível de caracteres utiliza n-gramas de 2 a 5 para capturar padrões morfológicos. Ambas as representações são normalizadas pela norma L2 para garantir comparabilidade entre amostras de comprimento variável.
- **Features Manuais (opcional).** Quando ativadas, são extraídas *features* estatísticas e estilométricas do texto original, incluindo comprimento em caracteres, número de palavras, número de sentenças, diversidade lexical,

pontuação e outros indicadores de estilo. Estas *features* são normalizadas (*z-score*) antes da concatenação com a representação TF-IDF.

DNN com PyTorch. O modelo DNN implementado com PyTorch utiliza entrada vetorial baseada na concatenação de TF-IDF de palavras e TF-IDF de caracteres. O pré-processamento segue o seguinte pipeline:

- **Vetorização TF-IDF de palavras.** Os textos de treino são transformados com `TfidfVectorizer`, configurado com `max_features=12000`, `gram_range=(1,3)`, `min_df=5`, `max_df=0.8` e sublinear TF. O mesmo vectorizador ajustado no treino é aplicado ao conjunto de avaliação.
- **Vetorização TF-IDF de caracteres.** Em paralelo, é ajustado um segundo `TfidfVectorizer` com `analyzer=char`, `max_features=12000`, `gram_range=(2,5)`, `min_df=5` e sublinear TF. As duas representações são concatenadas para estruturar a entrada final do classificador.
- **Conversão para tensores.** A matriz esparsa TF-IDF é convertida para representação densa por amostra e utilizada como entrada da DNN com PyTorch.

Transformer Genérico. O modelo Transformer utiliza um *generic tokenizer* para vocabulário compartilhado. O processamento inclui:

- **Tokenization com *AutoTokenizer*.** Utiliza-se um *tokenizer* pré-configurável (*AutoTokenizer* da biblioteca Transformers) que normaliza o texto, aplica *casefolding* e segmenta em sub-palavras ou *tokens*, segundo o esquema do *tokenizer* selecionado.
- **Codificação de Sequências.** As sequências de *tokens* são codificadas em índices do vocabulário, com comprimento máximo fixo de 512 *tokens*. Sequências mais curtas são preenchidas com o *token* de *padding* do *tokenizer*, e sequências mais longas são abreviadas.
- **Tensores de Atenção.** Com base no *tokenizer*, são construídas *attention masks* para indicar posições de *padding* ao modelo, permitindo que o Transformer ignore *tokens* de *padding* durante o cálculo de auto-atenção.

RoBERTa *Fine-tuned*. O modelo RoBERTa utiliza um *tokenizer* e configuração pré-treinados do modelo base. O pré-processamento específico para este modelo é o seguinte:

- **RoBERTa *Tokenizer*.** Utiliza o *tokenizer* específico do modelo pré-treinado RoBERTa-base [3], que normaliza a entrada e aplica *Byte-Pair Encoding* (BPE) tokenization. *Tokens* especiais do RoBERTa [3], como `<s>`, `</s>` e `<pad>`, são inseridos automaticamente conforme o formato esperado pelo modelo.

- **Sequências de Entrada.** O texto é convertido em sequências de *tokens* no formato de entrada do RoBERTa [3], com *tokens* especiais no início e no fim da sequência. As sequências são truncadas ou preenchidas até um comprimento máximo de 256 *tokens*, de acordo com a configuração de treino utilizada.
- **Embeddings Posicionais.** O modelo adiciona *embeddings* posicionais aos *embeddings* de *tokens* antes do processamento pelas camadas Transformer. No caso do RoBERTa [3], não são utilizados *token type embeddings* (*embeddings* de segmento) como no BERT clássico.

2.4 Formulação Matemática

Nesta subseção, sintetizam-se as principais formulações utilizadas no pré-processamento e no treino dos modelos.

TF-IDF e normalização para um termo t num documento d , com coleção D :

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}, \quad \text{idf}(t) = \log \left(\frac{1 + |D|}{1 + \text{df}(t)} \right) + 1$$

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

Após a vetorização, aplica-se normalização L2 por amostra:

$$\mathbf{x} \leftarrow \frac{\mathbf{x}}{\|\mathbf{x}\|_2}$$

Para *features* manuais, usa-se padronização *z-score* no treino:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j + \varepsilon}$$

DNNs (NumPy e PyTorch) Uma camada densa é dada por:

$$\mathbf{h}^{(l)} = \phi \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right)$$

com $\phi(\cdot) = \max(0, \cdot)$ (ReLU).

Na saída multiclasse:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^C \exp(z_j)}$$

A função de perda usada para classificação multiclasse é:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^C y_k \log p_k$$

No modelo PyTorch, a formulação é hierárquica: uma primeira *Binary Head* estima a origem do texto (humano ou gerado por IA), enquanto uma *Secondary Head* estima a *generator family*, condicionada às amostras identificadas como artificiais. O processo de treino otimiza simultaneamente uma perda binária e uma perda de entropia cruzada multiclasse sobre as famílias de IA.

Transformer e RoBERTa no mecanismo de auto-atenção:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

No Transformer genérico, após o *encoder*, usa-se *mean pooling* mascarado:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^T m_i \mathbf{h}_i}{\sum_{i=1}^T m_i}$$

onde $m_i \in \{0,1\}$ indica *tokens* válidos (não *padding*).

No RoBERTa *fine-tuned* [3], a *classification head* aplica projeção linear sobre a representação contextual e otimiza novamente a perda de entropia cruzada multiclasse.

3 Arquiteturas de Modelos

Esta secção descreve as arquiteturas efetivamente utilizadas nos quatro modelos principais do trabalho, com ênfase nas camadas, dimensões internas e estratégia de classificação multiclasse.

3.1 DNN com Numpy

O modelo DNN implementado de raiz com NumPy é uma rede *feedforward* totalmente ligada com três camadas ocultas, seguindo a topologia: input $\rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow C$, onde C representa o número de classes. Entre camadas densas são aplicadas ativações ReLU e regularização por *dropout* com taxas 0.2, 0.1 e 0.1. A camada de saída usa Softmax para obter distribuições de probabilidade por classe. Os pesos das camadas densas são inicializados com estratégia tipo He e incluem penalização L2.

3.2 DNN com PyTorch

O modelo DNN com PyTorch não replica exatamente a arquitetura NumPy. Em vez disso, utiliza um *backbone* denso partilhado seguido de dois *Classification Head*. O *backbone* é composto por: $\text{Linear}(d, 512) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(512, 256) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3)$.

Sobre esta representação intermédia, o modelo aplica: (i) um *Binary Head* $256 \rightarrow 128 \rightarrow 1$ para distinguir *Human* de *AI*; e (ii) um módulo de famílias $256 \rightarrow 128 \rightarrow (C - 1)$ para classificar Anthropic, Google, Meta e OpenAI. Esta formulação hierárquica altera a natureza da comparação com o NumPy DNN: ambos são modelos vetoriais densos, mas não são arquiteturalmente equivalentes.

3.3 Transformer Genérico

O classificador Transformer é *encoder-only*, composto por *embedding* de *tokens*, *embedding* posicional treinável e uma pilha de blocos *Transformer Encoder*. A configuração utilizada foi: comprimento máximo de sequência $T = 512$, dimensão do modelo $d_{\text{model}} = 512$, 16 cabeças de atenção, 6 camadas *encoder*, *dropout* 0.2 e activação GELU no *feedforward*. A representação final da sequência é obtida por *mean pooling* mascarado (ignorando *padding*), seguida de *dropout* e camada linear para *logits* multiclasse.

3.4 RoBERTa *Fine-tuned* (RoBERTa-base)

O quarto modelo utiliza *fine-tuning* de um *encoder* pré-treinado, concretamente RoBERTa-base [3], através de `AutoModelForSequenceClassification`. A arquitetura base possui 12 camadas Transformer, dimensão oculta 768 e 12 *attention heads*. Sobre o *encoder* pré-treinado é adicionada uma *classification head* para C classes. O treino ajusta *end-to-end* tanto o *encoder* como a *classification head*, permitindo transferir conhecimento linguístico pré-treinado para a tarefa de classificação de origem textual.

Table 1. Comparação de desempenho dos modelos no conjunto de avaliação.

Modelo	<i>Accuracy</i>	Macro-F1	<i>Macro-Precision</i>	<i>Macro-Recall</i>
DNN com NumPy	0.5500	0.5015	0.5950	0.5096
DNN com PyTorch	0.4800	0.4463	0.5974	0.4391
Transformer Genérico	0.5900	0.5097	0.6712	0.5456
RoBERTa <i>fine-tuned</i>	0.7900	0.7487	0.7892	0.7791

4 Treino e Avaliação

Todos os modelos foram treinados com divisão estratificada 80/20 entre treino e avaliação, preservando a proporção de classes. A avaliação baseou-se em métricas de

classificação multiclasse, com ênfase na *accuracy*, nos modelos baseados em *Trainer* (Transformer e RoBERTa) e no *F1-macro* para seleção do melhor *checkpoint*.

4.1 DNN com NumPy

O modelo DNN com NumPy foi treinado com *mini-batches* (*batch_size* = 32), até 100 épocas, taxa de aprendizagem inicial de 0.005 e *early stopping* com *patience* 5. A função de custo utilizada foi *Categorical Cross-Entropy*, com possibilidade de ponderação por classe para mitigar desequilíbrio residual. O processo de treino monitoriza a perda média por época, *accuracy* de treino e *accuracy* de validação, além de *accuracy* por classe no conjunto de avaliação.

Os resultados do treino podem ser visualizados na Fig. 2.

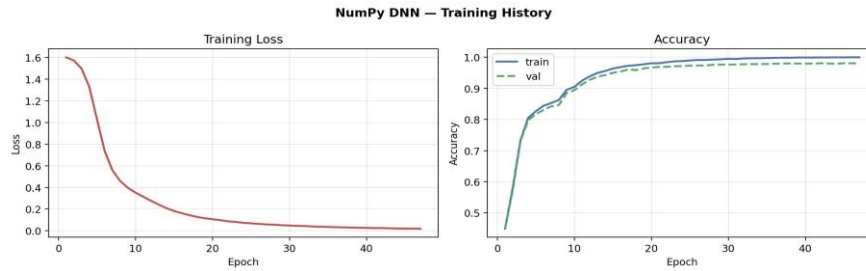


Fig.2. Resultados do treino do modelo DNN com NumPy

4.2 DNN com PyTorch

O modelo DNN com PyTorch foi treinado com o otimizador Adam, usando *learning_rate* = 0,01, *batch_size* = 32 e até 10 épocas. A função objetivo combina *BCEWithLogitsLoss* para a *binary head Human vs. AI* com *Cross-Entropy Loss* para a *classification head* das *generator families*. Foi aplicado *early stopping* com *patience* 3, através da monitorização da métrica de validação por época. Para análise complementar, foram calculadas métricas globais no conjunto de avaliação e *accuracy* por classe.

Os resultados do treino podem ser visualizados na Fig. 3.

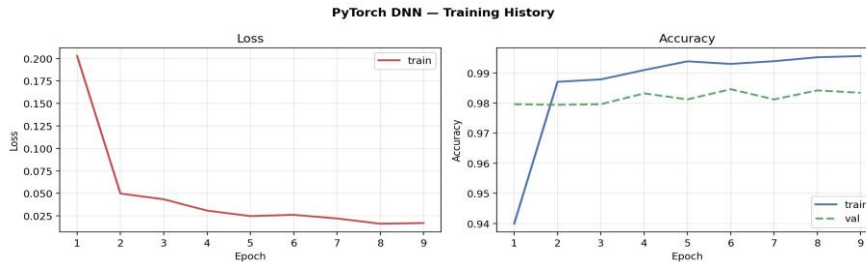


Fig.3. Resultados do treino do modelo DNN com PyTorch.

4.3 Transformer Genérico

O Transformer genérico foi treinado com a API *Trainer* da biblioteca Transformers, com os seguintes hiperparâmetros principais: 3 épocas, $batch_size = 32$, $learning_rate = 0,002$, $weight_decay$ de 0.01, $warmup_ratio$ de 0.1 e $scheduler$ cosseno. A avaliação foi executada ao fim de cada época, com retenção automática do melhor *checkpoint* Segundo o *F1-macro*.

Os resultados do treino podem ser visualizados na Fig. 4.



Fig.4. Resultados do treino do modelo Transformer Genérico.

4.4 RoBERTa *Fine-tuned*

O modelo RoBERTa-base [3] foi ajustado por *fine-tuning* com *Trainer*, utilizando 3 épocas, $batch_size = 32$, $learning_rate = 0,0002$, $weight_decay$ de 0.01, comprimento máximo de sequência 256 e acumulação de gradientes em 2 passos. A avaliação foi realizada por época, com carregamento automático do melhor modelo no final do treino segundo o *F1-macro*.

Em termos comparativos, a avaliação final considerou consistência entre desempenho global ($accuracy/F1-macro$) e estabilidade por classe, permitindo analisar o compromisso entre modelos mais leves (DNNs) e modelos baseados em Transformer.

Os resultados do treino podem ser visualizados na Fig. 5.

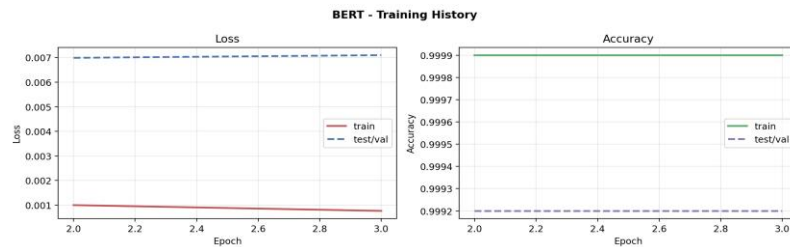


Fig.5. Resultados do treino do modelo RoBERTa *fine-tuned*.

5 Discussão

5.1 Hierarquia de Desempenho: Vetorial vs. Contextual

Os resultados revelam uma hierarquia clara entre os quatro modelos avaliados, estratificada segundo o nível de complexidade arquitetural e o tipo de representação textual utilizado. O modelo RoBERTa-base, pré-treinado e ajustado por *fine-tuning*, alcançou o melhor desempenho com *accuracy* de 0.7900 e *F1*macro de 0.7487, materializado em *macro-precision* de 0.7892 e *macro-recall* de 0.7791. Este resultado coloca-se aproximadamente 29 pontos percentuais acima do Transformer genérico (*F1*: 0.5097) e 44 pontos percentuais acima dos modelos dendríticos vetoriais (*F1* NumPy: 0.3171), sinalizando que o conhecimento linguístico pré-existente capturado durante o treino em larga escala é essencial para esta tarefa.

5.2 Ganho do Mecanismo de Atenção em Relação a Representações Vetoriais

O Transformer genérico treinado de raiz, atinge uma performance intermediária com *accuracy* de 0.5900 e *F1-macro* de 0.5097. Este desempenho demonstra que o mecanismo *self-attention*, mesmo sem inicialização pré-treinada, oferece vantagem substancial sobre representações TF-IDF aliadas a camadas densas. A diferença de aproximadamente 19 pontos percentuais em *F1*-macro entre o Transformer genérico e o DNN com NumPy sugere que a capacidade de capturar dependências contextuais, mesmo num espaço de vocabulário limitado e com inicialização aleatória, supera significativamente a abordagem de extração estática de *n*-gramas. Porém, o desempenho inferior ao RoBERTa-base (diferença de 24 pontos em *F1*-macro) sublinha que, para classificação de origem textual, o conhecimento pré-treinado em grandes corpora é preferível aos mecanismos de atenção treinados de raiz num *dataset* de dimensão moderada.

5.3 Análise Comparativa dos Modelos Vetoriais: DNN com NumPy vs. DNN com PyTorch

Entre os modelos baseados em representações vetoriais TF-IDF, o DNN com NumPy superou o DNN com PyTorch, com *accuracy* de 0.3700 contra 0.3300 e *F1-macro* de 0.3171 contra 0.2854. Embora ambos utilizem entrada TF-IDF, os dois modelos diferem na arquitetura e formulação da tarefa. O DNN com NumPy implementa uma rede totalmente ligada com três camadas ocultas e saída multiclases direta, enquanto o DNN com PyTorch adota uma formulação hierárquica com uma *binary head* para *Human vs. AI* e uma *secondary head* para classificação das *generator model family*. A

superioridade relativa do DNN com NumPy pode refletir (i) maior estabilidade da formulação multiclass direta numa tarefa desafiadora, ou (ii) diferenças em regularização, taxa de aprendizagem ou inicialização de pesos. Ambos os modelos revelam limitações fundamentais das abordagens vetoriais estáticas: a representação TF-IDF, ainda que beneficiada de n-gramas de caracteres, não captura padrões semânticos profundos que distinguem textos humanos de sintéticos. Os resultados sugerem que a sequência, o contexto imediato e as dependências de longo alcance são sinais críticos para esta tarefa.

5.4 Implicações para o *Trade-off* Computacional e de Performance

A comparação dos quatro modelos representa o compromisso fundamental entre custo computacional e desempenho. Os modelos vetoriais (DNN com NumPy e DNN com PyTorch) são economicamente vantajosos em termos de latência de treino e espaço de parâmetros (~ 13k e 50k pesos, respetivamente), mas oferecem capacidade representacional insuficiente para discriminar robustamente a origem textual. O Transformer genérico incrementa complexidade (~ 70M parâmetros) e custo de treino, mas demonstra melhoria considerável de 14 pontos percentuais em *F1-macro* em relação ao melhor modelo vetorial. O RoBERTa-base, embora requerer treino *end-to-end* em pesos pré-treinados (~ 125M parâmetros), oferece o melhor custo-benefício: não requer ajuste extensivo de hiperparâmetros e converge rapidamente para uma performance superior à dos Transformer treinados de raiz. Isto sugere que, para classificação de origem textual, o investimento em computação para *fine-tuning* de modelos pré-treinados é mais eficiente que treinar modelos de grande dimensão de raiz.

5.5 Análise por Classe e Robustez

Para além das métricas agregadas, é essencial analisar a variância de desempenho entre classes. O RoBERTa-base mantém *macro-recall* de 0.7791, evidenciando que o desempenho não é controlado por classes dominantes, sugerindo equilíbrio entre sensibilidade e especificidade em todas as categorias. Em contraste, os modelos vetoriais podem sofrer degradação numa ou duas classes, sendo ocultada pelas médias macroscópicas. Esta observação reforça que a seleção de métricas apropriadas e a análise estratificada por classe são essenciais em tarefas multiclass desbalanceadas.

6 Limitações e Ameaças à Validade

Este estudo compara arquiteturas diversas para classificação de origem textual, mas enfrenta limitações relevantes que restringem a generalização das conclusões.

Validade Interna. (i) A avaliação foi conduzida com uma única partição estratificada (80/20), enviesando potencialmente a estimacão da performance real. Validação cruzada com múltiplas sementes permitiria quantificar a variância estrutural; (ii) Os modelos vetoriais e contextuais não partilham a mesma representacão de entrada, complicando a comparabilidade direta: os vetoriais usam TF-IDF (estático, sem contextualizacão), enquanto os contextuais usam *tokenizers* pré-treinados que capturam semântica e topologia lexical. Esta heterogeneidade é inerente à comparacão, mas limita conclusões sobre origem arquitetural vs. representacional das diferenças de desempenho; (iii) Os hiperparâmetros foram ajustados independentemente para cada modelo; é possível que *fine-tuning* conjunto levasse a conclusões distintas; (iv) O Transformer genérico foi inicializado com pesos aleatórios e treinado num contexto de dados limitado; investigacão futura com pré-treino em corpora de origem textual poderia modificar a hierarquia observada.

Validade Externa. Os resultados podem não generalizar para: (i) domínios textuais fora de conteúdo jornalístico e académico (e.g., redes sociais, narrativa jurídica ou biomédica); (ii) modelos geradores não testados (e.g., GPT-4, Claude, Gemini, ou futuras arquiteturas); (iii) cenários multilíngues ou misturas de idioma; (iv) textos modificados através de *paraphrase*, ofuscação ou ataque adversarial. A tarefa estudada reflete um contexto específico (deteção de origem em 2 + 4 classes), e a *performance* em cenários de *zero-shot* ou adaptacão a novos geradores permanece desconhecida.

Ameaças Específicas. (i) Desequilíbrio residual de classes pode favorecer classes maioritárias nalguns modelos apesar do balanceamento aplicado; (ii) O artigo não reporta variância de estimacão (desvio padrão ou intervalos de confiança), limitando a interpretacão de diferenças entre modelos; (iii) Não foi realizado teste de significância estatística, pelo que diferenças de alguns pontos percentuais podem não ser estatisticamente robustas.

7 Conclusão e Trabalho Futuro

7.1 Síntese e Implicações Metodológicas

Este trabalho apresentou uma comparacão sistemática de quatro arquiteturas para classificacão de origem textual, desde modelos vetoriais leves (DNN com NumPy e DNN com PyTorch) até modelos contextuais avançados (Transformer genérico e RoBERTa-base). A hierarquia de desempenho observada - RoBERTa-base (F1: 0.7487) > Transformer genérico (F1: 0.5097) > DNN com NumPy (F1: 0.3171) > DNN com PyTorch (F1: 0.2854) - ilustra um princípio fundamental na classificacão de texto: a superioridade das arquiteturas contextuais. Quando suportadas por pré-treino massivo, estas superam por uma margem significativa as abordagens baseadas em vetores estáticos. Porém, o desempenho do Transformer genérico (F1: 0.5097) evidencia que mecanismos de atencão, mesmo não pré-treinados, oferecem ganho considerável sobre

as representações TF-IDF, sugerindo que a captura de dependências e contexto local é mais importante do que a escala de pré-treino para este trabalho.

A comparação entre os dois modelos vetoriais - com o DNN com NumPy a superar o DNN com PyTorch em 3,17 pontos de *F1-macro*, apesar da sua arquitetura menos sofisticada - levanta questões sobre fatores não-arquiteturais. Elementos como a regularização, a taxa de aprendizagem efetiva ou as propriedades da formulação multiclasse direta face à hierárquica revelam-se determinantes. Isto sublinha que a configuração do treino e a própria definição da tarefa são tão críticas para o sucesso quanto a escolha da arquitetura.

7.2 Contribuições para Detecção Textual e NLP

As conclusões extrapolam para cenários práticos de deteção de origem textual:

Para aplicações com restrições computacionais. A formulação DNN vetorial oferece latência aceitável e requisitos modestos de memória, mas fornece uma fiabilidade reduzida ($F1 < 0.32$). Um sistema com restrições severas deveria preferir o Transformer genérico como ponto de partida.

Para aplicações de precisão crítica. O RoBERTa-base (ou modelos equivalentes pré-treinados) é o recomendado, oferecendo um *F1-macro* de 0.7487 com uma margem confortável face a modelos treinados de raiz. O custo computacional de implementação é moderado (<250 ms de latência em hardware moderno).

Para exploração investigativa. O Transformer genérico oferece o ponto de equilíbrio: performance quase 5 vezes superior aos modelos vetoriais, com a possibilidade de interpretação dos mecanismos de atenção e sem dependência de pré-treino corporativo.

7.3 Trabalho Futuro

Recomendamos as seguintes direções de investigação:

Validação cruzada com multiplicidade. Implementar validação cruzada (5 ou 10 *folds*) com múltiplas sementes, quantificando a variância de estimação e o intervalo de confiança de cada métrica. Isto permitirá testes de significância e reduzirá o enviesamento por partição única.

Análise de erros estratificada. Desagregar erros por classe, por comprimento de sequência e por marcas linguísticas (exemplo: presença de citações, estrutura formal). Isto revelará padrões de confusão e debilidades específicas.

Investigação de representações comuns. Comparar modelos sob uma idêntica representação de entrada (exemplo: todos com *token-embeddings* pré-treinados do RoBERTa-base). Isto isolaria os efeitos arquiteturais dos efeitos representacionais.

Pré-treino adaptado. Pré-treinar um Transformer de menor escala (~30M parâmetros) num *corpus* de origem textual diversificado (misturas de *Human*, GPT-2, GPT-3, etc.), avaliando depois o desempenho *zero-shot* e *few-shot* em novos geradores.

Técnicas de robustez. Testar a resiliência *adversarial examples*, paráfrases e misturas linguísticas. Avaliar a degradação contínua conforme os textos se afastam da distribuição de treino.

Ensemble adaptativo. Investigar combinações ponderadas dos modelos (ex: Transformer genérico para deteção rápida + RoBERTa para refinamento), equilibrando latência e exatidão.

Investigação de calibração. Estudar a fiabilidade das probabilidades preditas por cada modelo, utilizando técnicas de calibração como *Platt scaling* ou *temperature scaling*.

Em conclusão, os *embeddings* pré-treinados e os mecanismos de atenção são ferramentas indispensáveis para uma classificação de origem textual robusta. Contudo, o desempenho ainda deixa uma margem significativa para investigações futuras em robustez, adaptação *cross-domain* e eficiência computacional.

References

1. Harris, C.R., Millman, K.J., van der Walt, S.J., et al.: Array programming with NumPy. *Nature* 585(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>
2. Paszke, A., Gross, S., Massa, F., et al.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: Wallach, H., et al. (eds.) *Advances in Neural Information Processing Systems* 32, pp. 8024–8035. Curran Associates, Inc. (2019)
3. Liu, Y., Ott, M., Goyal, N., et al.: RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019)

4. MLNTeam-Unical: OpenTuringBench. Hugging Face Datasets (2024).
<https://huggingface.co/datasets/MLNTeam-Unical/OpenTuringBench>. Último acesso em 2026/03/29
5. Dahoas: instruct-synthetic-prompt-responses. Hugging Face Datasets (2023).
<https://huggingface.co/datasets/Dahoas/instruct-synthetic-prompt-responses>. Último acesso em 2026/03/29
6. artem9k: ai-text-detection-pile. Hugging Face Datasets (2023).
<https://huggingface.co/datasets/artem9k/ai-text-detection-pile>. Último acesso em 2026/03/29
7. Anthropic: persuasion. Hugging Face Datasets (2023).
<https://huggingface.co/datasets/Anthropic/persuasion>. Último acesso em 2026/03/29